

Лабораторная работа по MySQL №4

Для начала создаем базу данных и таблицу. Входим в командную строку MySQL, и выполняем команды:

```
CREATE DATABASE mydb;

CREATE TABLE employees
(id tinyint(4) DEFAULT '0' NOT NULL AUTO_INCREMENT,
first varchar(20), last varchar(20), address varchar(255),
position varchar(50), PRIMARY KEY (id), UNIQUE id (id));

INSERT INTO employees VALUES
(1,'Bob','Smith','128 Here St, Cityname','Marketing Manager');
INSERT INTO employees VALUES
(2,'John','Roberts','45 There St ,Townville','Telephonist');
INSERT INTO employees VALUES
(3,'Brad','Johnson','1/34 Nowhere Blvd, Snowston','Doorman');
```

В результате будет создана база данных `mydb`. В ней будет создана таблица `employees` (работники). И в эту таблицу будут вставлены три записи с данными о работниках. Для экспериментов с PHP и MySQL этого вполне достаточно.

Давайте выведем эти данные из базы данных в HTML-страницу. Для общения с MySQL из PHP понадобятся следующие функции.

`int mysql_connect(string hostname, string username, string password);` — создать соединение с MySQL.

Параметры:

`Hostname` — имя хоста, на котором находится база данных.

`Username` — имя пользователя.

`Password` — пароль пользователя.

Функция возвращает параметр типа `int`, который больше 0, если соединение прошло успешно, и равен 0 в противном случае.

`int mysql_select_db(string database_name, int link_identifier);` — выбрать базу данных для работы.

Параметры:

`Database_name` — имя базы данных.

`link_identifier` — ID соединения, которое получено в функции `mysql_connect`. (параметр необязательный, если он не указывается, то используется ID от последнего вызова `mysql_connect`)

Функция возвращает значение `true` или `false`

`int mysql_query(string query, int link_identifier);` — функция выполняет запрос к базе данных.

Параметры:

`query` — строка, содержащая запрос

`link_identifier` — см. предыдущую функцию.

Функция возвращает ID результата или 0, если произошла ошибка.

`int mysql_result(int result, int i, column);` — функция возвращает значение поля в столбце `column` и в строке `i`.

`int mysql_close(int link_identifier);` — функция закрывает соединение с MySQL.

Параметры: `link_identifier` — см. выше.

Функция возвращает значение `true` или `false`

Создайте файл с расширением `.php` и наберите в нем следующий текст:

```
<html>
<body>
  <?php
    $db = mysql_connect("localhost", "root");
```

```

mysql_select_db("mydb",$db);

$result = mysql_query("SELECT * FROM employees",$db);

printf("First Name: %s<br>\n", mysql_result($result,0,"first"));
printf("Last Name: %s<br>\n", mysql_result($result,0,"last"));
printf("Address: %s<br>\n", mysql_result($result,0,"address"));
printf("Position: %s<br>\n", mysql_result($result,0,"position"));
mysql_close($db);
?>
</body>
</html>

```

Теперь рассмотрим построчно, что происходит в этой программе. Функция `mysql_connect()` открывает связь с сервером баз данных MySQL. В качестве параметров мы указываем ей имя узла (`localhost`), на котором находится база данных, имя пользователя (`root`), под которым мы будем с ней работать, и пароль (в данном случае он пустой и потому не указывается).

Имя узла `localhost` означает, что сервер MySQL находится на той же машине, что и сам Web-сервер с PHP - движком. В принципе ничто не мешает вам (имея права) обратиться к серверу MySQL, который находится на соседней машине или вообще на другом конце земного шара.

В результате выполнения этой функции получаем некое значение, которое присваиваем переменной `$db`. Эта переменная называется идентификатором соединения (см. выше описание синтаксиса).

Соединившись с сервером выбираем базу данных, с которой будем работать (ведь на одном и том же сервере могут "крутиться" несколько баз данных). Это делается с помощью функции `mysql_select_db()`. В качестве параметров мы передаем название нужной нам базы данных и идентификатор соединения, полученный нами при выполнении предыдущей команды.

В результате выполнения функции `mysql_select_db()` мы получаем значение `true` или `false`. Если соединение с базой данных произошло успешно — `true`, если нет — `false`. Для того, чтобы наша программа-страница работала более интеллектуально, мы можем при желании проанализировать возвращаемое значение и если оно будет `false`, вывести хорошее информативное сообщение об ошибке.

И наконец мы обращаемся к базе данных с запросом, написанным на языке SQL. Для этого служит функция `mysql_query()`. В качестве первого параметра мы передаем текст запроса, а в качестве второго — передаём идентификатор, полученный от выполнения функции `mysql_connect()`.

Результаты выполнения функции `mysql_query()` — записи, удовлетворяющие нашему запросу - помещаем в переменную `$result`.

И наконец, с помощью функции `mysql_result()` извлекаем из результатов выполнения нашего запроса (т.е. переменной `$result`), первый ряд-запись (который имеет порядковый номер 0), и значение определенного поля (по его имени). Перебирая друг за другом записи от 0 до 2, мы выберем все записи, что хранятся в нашей маленькой базе данных.

В данном коде в полной мере используется функция `printf()`. Если говорить коротко, то в каждой приведенной выше строке комбинация символов `%s` обозначает, что вместо нее должна быть поставлена переменная, идущая во второй части выражения `printf`. Причем эта переменная должна быть переведена в тип "строковая переменная". Более подробное описание синтаксиса функции `printf()` читайте в руководстве по языку.

Далее мы познакомимся с более интеллектуальными, чем `mysql_result()`, функциями выборки данных из БД `mysql_fetch_row()` и `mysql_fetch_array()`, В дальнейшем мы рекомендуем пользоваться именно ими, как более быстрыми и удобными, чем `mysql_result()`.

Вывод данных из базы данных

Давайте теперь попробуем вывести все записи, хранящиеся в нашей базе данных. Обратимся к базе данных со следующим кодом:

```
<html>
```

```

<body>
<?php
$db = mysql_connect("localhost", "root");
mysql_select_db("mydb",$db);

$result = mysql_query("SELECT * FROM employees",$db);

echo "<table border=1>\n";
echo "<tr><td>Name</td><td>Position</td></tr>\n";

while ($myrow = mysql_fetch_row($result))
{
    printf("<tr><td>%s %s</td><td>%s</td></tr>\n", $myrow[1],
        $myrow[2], $myrow[3]);
}

echo "</table>\n";
?>
</body>
</html>

```

Вы вероятно заметили, что в данном коде вы ввели несколько новых функций и конструкций. Наиболее очевидной из них является цикл `while()`. Цикл говорит, что до тех пор, пока в переменной `$result` остается запись для выборки, ее необходимо извлечь с помощью функции `mysql_fetch_row` и присвоить переменной `$myrow`. А после этого выполнить код, что расположен внутри фигурных скобок. Приглядитесь к коду внимательнее и разберитесь в этой конструкции.

Для понимания этого кода разберем понятие "массив". Выполнение функции `mysql_query` дает в результате массив, который хранится в переменной `$result`. Если представить этот массив схематически, то он будет выглядеть как:

0 id 1 first 2 last 3 address 4 position Порядковый номер элемента массива 1 Bob Smith 128 Here St, Cityname Marketing Manager 0 2 John Roberts 45 There St ,Townville Telephonist 1 3 Brad Johnson 1/34 Nowhere Blvd, Snowston Doorman 2

Переменная `$result` является массивом. Причем не простым массивом, а двумерным. В нем содержатся три строки с номерами от 0 до 2. каждая из которых содержит 5 столбцов от 0 до 4. Для того, чтобы вывести на странице все записи, нам надо пройти от 0-й строчки массива до 2-й. Лучше всего это делать в цикле с помощью функции `mysql_fetch_row` (которая в переводе буквально означает – "выбрать ряд").

Функции `mysql_fetch_row` в качестве параметра подается массив `$result`. Функция выбирает из него строку, которую мы можем записать в переменную `$myrow` и автоматически переходит на следующую строку. Вызвав снова `mysql_fetch_row`, выберем следующую строку из массива, и так далее до тех пор, пока не достигнем конца массива. В этом случае `mysql_fetch_row` вернет значение `false`, которое послужит сигналом, что все записи выбраны и можно завершить цикл.

Теперь стоит задача как-то вывести в теле цикла полученную запись. Выбранный ряд хранится в переменной `$myrow`. Она также, как и `$result`, является массивом, только одномерным. Схематически это выглядит как:

0 id 1 first 2 last 3 address 4 position Порядковый номер элемента массива 1 Bob Smith 128 Here St, Cityname Marketing Manager 0

А вот как будет выглядеть содержимое переменной `$myrow` при втором прохождении цикла: 0 id 1 first 2 last 3 address 4 position Порядковый номер элемента массива 2 John Roberts 45 There St ,Townville Telephonist 1

При третьем прохождении — как 0 id 1 first 2 last 3 address 4 position Порядковый номер элемента массива 3 Brad Johnson 1/34 Nowhere Blvd, Snowston Doorman 2

К каждому столбцу в массиве `$myrow` мы можем обратиться по его порядковому номеру, который заключается в квадратные скобки. Например, в первом цикле, `$myrow[1]` равно `Bob`, во втором `$myrow[4]` равно `"Telephonist"`.

На первый взгляд процедура извлечения данных весьма сложна, но она вполне логически понятна и объяснима. Главное не забывайте, что элементы массивов нумеруются от 0, а не от 1.

Далее, вывод переменных в HTML с помощью функции `printf()` — дело техники, уже знакомой нам по предыдущему примеру.

Наш код содержит недостаток: если в базе данных не будут найдены записи, удовлетворяющие нашему запросу, мы не получим никакого сообщения об этом. Неплохо было бы, чтобы программа выдавала какое-нибудь сообщение. Сделаем ее более дружелюбной.

Взгляните на следующий код:

```
<html>
<body>
  <?php
    $db = mysql_connect("localhost", "root");
    mysql_select_db("mydb",$db);

    $result = mysql_query("SELECT * FROM employees",$db);
    if ($myrow = mysql_fetch_array($result))
    {
      echo "<table border=1>\n";
      echo "<tr><td>Name</td><td>Position</td></tr>\n";

      do{
        printf("<tr><td>%s %s</td><td>%s</td>\n", $myrow["first"], $myrow["last"],
          $myrow["address"]);
      } while ($myrow = mysql_fetch_array($result));

      echo "</table>\n";
    } else
    {
      echo "Sorry, no records were found!";
    }
  ?>
</body>
</html>
```

В данном коде мы опять ввели некоторые новые понятия, но они достаточно просты. Во-первых, вместо функции `mysql_fetch_row()` мы использовали функцию `mysql_fetch_array()`. Она работает точно так же, как и `mysql_fetch_row()` за одним замечательным исключением: с помощью этой функции мы можем обращаться к каждому полю массива не по номеру, а по имени.

Например, если раньше для получения имени нам приходилось писать `$myrow[1]` (1 — второй столбец массива), то теперь мы можем писать `$myrow["first"]` ("first" — название столбца в базе данных и в массиве). Второй вариант естественно гораздо информативнее и удобнее.

Кроме этого, в коде использован цикл `do/while` и условная конструкция `if-else`. Выражение `if-else` говорит, что если мы можем присвоить значение `$myrow`, то надо начать выборку, в противном случае мы понимаем, что записей нет, переходим к блоку `else` и выводим соответствующее сообщение.

Чтобы проверить, как работает эта часть кода, замените SQL-выражение на

```
SELECT * FROM employees WHERE id=6
```

или на какое-нибудь другое, которое не даст результата.

Цикл `do/while` — это всего лишь вариант цикла `while()`, который мы использовали в предыдущем примере. Мы обратились за помощью к циклу `do/while` по одной простой причине. В конструкции `if` мы уже сделали выборку первого ряда и присвоили его переменной `$myrow`.

Если бы мы сейчас воспользовались прежней конструкцией, т.е. `while`

```
$myrow = mysql_fetch_row($result)
```

мы бы затерли значения первой выбранной записи, заменив ее значениями второй записи. В случае же с циклом `do/while` мы проверяем условие после того, как код цикла выполнится по крайней мере один раз. Таким образом, ни одна запись не ускользнет из наших рук.

А сейчас давайте сделаем код в цикле и `if-else` конструкцию еще более красивым.

Создаем ссылки на лету

Сейчас поработаем с параметрами запроса. Как вы уже наверняка знаете, существует три способа передачи параметров запроса. Первый, использовать метод `GET` в форме. Второй — набрать параметры прямо в адресной строке браузера. И третий, это вставить параметры в обычную ссылку на странице. То есть сделать ссылку примерно такого вида

```
<a href="http://my_machine/mypage.php3?id=1">
```

Научимся создавать такие ссылки на лету.

Для начала, давайте обратимся к базе данных и выведем список персонала. Взгляните на следующий код. Многое в нем вам будет знакомо.

```
<html>
<body>
  <?php
    $db = mysql_connect("localhost", "root");
    mysql_select_db("mydb",$db);
    $result = mysql_query("SELECT * FROM employees",$db);
    if ($myrow = mysql_fetch_array($result))
    {
      do{
        printf("<a href=\"%s?id=%s\">%s %s</a><br>\n",
          $PHP_SELF, $myrow["id"],$myrow["first"],
          $myrow["last"]);
      }while ($myrow = mysql_fetch_array($result));
    }else
    {
      echo "Sorry, no records were found!";
    }
  ?>
</body>
</html>
```

Все вам должно быть знакомо, кроме функции `printf()`. Поэтому, давайте рассмотрим ее поближе. Во-первых, обратите внимание на обратные косые черты. Они говорят РНР-движку, что необходимо вывести символ, следующий за чертой, а не рассматривать его, как служебный символ или как часть кода. В данном случае это касается кавычки, которая нам нужна в тексте ссылки, но для РНР является символом окончания текстовой строки.

Далее, в коде используется интересная переменная `$PHP_SELF`. В этой переменной всегда хранится имя и URL текущей страницы. В данном случае эта переменная важна для нас потому, что мы хотим через ссылку вызвать страницу из нее самой. То есть вместо того, чтобы делать две страницы, содержащие разные коды для разных действий, мы все действия записали в одну страницу. С помощью условий `if-else` мы будем переводить стрелки с одного кода на другой, гоняя одну и ту же страницу по кругу. Это конечно увеличит размер страницы и время, необходимое на ее обработку, но в некоторых случаях, такой трюк очень удобен.

Переменная `$PHP_SELF` гарантирует нам, что наша страница будет работать даже в том случае, если мы перенесем ее в другой каталог или даже на другую машину.

Ссылки, сгенерированные в цикле ссылаются на ту же самую страницу, только к имени самой страницы на лету добавлена некоторая информация: переменные и их значения.

Переменные, которые передаются в строке-ссылке, автоматически создаются РНР-движком, и к ним можно обращаться так, как если бы вы их создавали в коде сами. При втором проходе страницы

наша программа отреагирует на эти пары name=value и направит ход исполнения на другие рельсы. В данном случае мы проверим, есть ли переменная \$id, и в зависимости от результата выполним тот или иной код. Вот как это будет выглядеть:

```
<html>
<body>
  <?php
    $db = mysql_connect("localhost", "root");
    mysql_select_db("mydb",$db); // display individual record

    if ($id)
    {
        $result = mysql_query("SELECT * FROM employees WHERE id=$id",$db);

        $myrow = mysql_fetch_array($result);
        printf("First name: %s\n<br>", $myrow["first"]);
        printf("Last name: %s\n<br>", $myrow["last"]);
        printf("Address: %s\n<br>", $myrow["address"]);
        printf("Position: %s\n<br>", $myrow["position"]);
    }else
    {
        // show employee list
        $result = mysql_query("SELECT * FROM employees",$db);

        if ($myrow = mysql_fetch_array($result))
        {
            // display list if there are records to display
            do{
                printf("<a href=\"%s?id=%s\">%s %s</a><br>\n", $PHP_SELF, $myrow["id"],
                    $myrow["first"], $myrow["last"]);
            }while ($myrow = mysql_fetch_array($result));
        }else
        {
            // no records to display
            echo "Sorry, no records were found!";
        }
    }
  ?>
</body>
</html>
```

Код усложнился, поэтому мы добавили в него некоторые комментарии, чтобы он стал яснее. Для однострочных комментариев можно использовать символы //. Если комментарий нужно уместить на нескольких строках, используйте скобки /* ... */.

Итак, вы наконец создали действительно полезную PHP-страницу, работающую с MySQL.

Сохранение данных в базе данных

Мы научились извлекать данные из базы и выводить их на странице. Теперь давай попробуем осуществить обратное действие. С PHP это не составит большого труда.

Создадим простую форму:

```
<html>
<body>
  <form method="post" action="<?php echo $PHP_SELF?>">
    First name:<input type="Text" name="first"><br>
```

```

        Last name:<input type="Text" name="last"><br>
        Address:<input type="Text" name="address"><br>
        Position:<input type="Text" name="position"><br>
        <input type="Submit" name="submit" value="Enter information">
    </form>
</body>
</html>

```

Обратите внимание, мы опять используем переменную `$PHP_SELF`. Как мы уже сказали, PHP-код можно как угодно перемешивать с обычным HTML. Также обратите внимание, что название каждого элемента формы совпадает с названием поля в базе данных.

Вообще-то, это не обязательно, но весьма удобно, чтобы в дальнейшем не запутаться в том, какая переменная какому полю в базе данных соответствует.

Помимо этого мы присвоили имя кнопке Submit. Это сделано для того, чтобы в коде затем проверить, есть ли переменная `$submit`. Таким образом, когда страница будет вызываться, мы будем узнавать, вызывается ли она в первый или во второй раз.

Следует еще раз отметить, что вовсе не обязательно писать код так, чтобы страница снова и снова вызывала саму себя. Программу можно разделить на две, три и более страниц, если угодно. Но при таком подходе, вся программа находится в одном файле, а это бывает весьма удобно.

Итак, давайте добавим код, который будет проверять, введены ли в форму данные. Пока это будет лишь простая проверка, при которой все переменные, передаваемые странице, будут выводиться на экран с помощью переменной `$HTTP_POST_VARS`. Эта переменная удобна в случае отладки. Если вы хотите вывести на экран вообще все переменные, используемые в странице, вызовите переменную `$GLOBALS`.

```

<html>
<body>
    <?php
        if ($submit)
        {
            // process form
            while (list($name, $value) = each($HTTP_POST_VARS)) {
                echo "$name = $value<br>\n";
            }
        } else
        {
            // display form
        }
    ?>

    <form method="post" action="<?php echo $PHP_SELF?>">
        First name:<input type="Text" name="first"><br>
        Last name:<input type="Text" name="last"><br>
        Address:<input type="Text" name="address"><br>
        Position:<input type="Text" name="position"><br>
        <input type="Submit" name="submit" value="Enter
            information">
    </form>

    <?php
        } // end if
    ?>
</body>
</html>

```

Теперь давайте возьмем переданную через форму информацию и внесем ее в базу данных.

```

<html>

```

```

<body>
  <?php
  if ($submit)
  {
    // process form
    $db = mysql_connect("localhost", "root");
    mysql_select_db("mydb",$db);

    $sql = "INSERT INTO employees (first,last,address,position)
    VALUES ('$first','$last','$address','$position')";

    $result = mysql_query($sql);

    echo "Thank you! Information entered.\n";
  } else{
    // display form
  ?>

  <form method="post" action="<?php echo $PHP_SELF?>">
    First name:<input type="Text" name="first"><br>
    Last name:<input type="Text" name="last"><br>
    Address:<input type="Text" name="address"><br>
    Position:<input type="Text" name="position"><br>
    <input type="Submit" name="submit" value="Enter information">
  </form>
  <?php
  }
  // end if
  ?>
</body>
</html>

```

Мы внесли данные в базу. Тем не менее наш код далек от идеального. Что случится, если при заполнении формы кто-то оставит пустые поля или введет текст в поле, в которое надо ввести число? Что произойдет, если в поданных данных будет ошибка?

Ранее мы записывали SQL-выражение в переменную (`$sql`), прежде чем передать запрос в базу данных через функцию `mysql_query()`. Это делается на случай отладки. Если что-то пойдет не так, мы всегда сможем вывести интересующее нас SQL-выражение на экран и проверить, нет ли в нем ошибок.

Мы уже знаем, как вставлять данные в базу. Теперь давайте научимся менять записи, которые уже находятся в таблице.

Редактирование данных сочетает в себе два кода. Которые мы уже проходили: извлечение данных из базы с выводом их на экран, и внесение данных через форму обратно в базу. Тем не менее программа правки данных немного отличается тем, что мы в форме должны вывести некую конкретную запись. Для начала давайте воспользуемся кодом для вывода списка служащих на экран. Однако теперь информацию о служащих мы будем отображать в форме. Код страницы будет выглядеть так:

```

<html>
<body>
  <?php
    $db = mysql_connect("localhost", "root");

    mysql_select_db("mydb",$db);

```

```

        if ($id) {
            // query the DB
            $sql = "SELECT * FROM employees WHERE id=$id";
            $result = mysql_query($sql);
            $myrow = mysql_fetch_array($result);
        }

        <form method="post" action="<?php echo $PHP_SELF?>">
            <input type="hidden" name="id" value="<?php echo
            $myrow["id"] ?>">
            First name:<input type="Text" name="first" value="<?php
            echo $myrow["first"] ?>"><br>
            Last name:<input type="Text" name="last" value="<?php
            echo $myrow["last"] ?>"><br>
            Address:<input type="Text" name="address" value="<?php
            echo $myrow["address"] ?>"><br>
            Position:<input type="Text" name="position" value="<?php
            echo $myrow["position"] ?>"><br>
            <input type="Submit" name="submit" value="Enter
            information">
        </form>

        <?php
    } else {
        // display list of employees
        $result = mysql_query("SELECT * FROM employees",$db);
        while ($myrow = mysql_fetch_array($result)) {
            printf("<a href=\"%s?id=%s\">%s %s</a><br>\n", $PHP_SELF,
                $myrow["id"], $myrow["first"],
                $myrow["last"]);
        }
    }
    ?>
</body>
</html>

```

В этой странице мы просто вывели в каждое поле формы соответствующее значение из базы данных. Теперь усложним программу. Добавим к ней возможность внесения отредактированных данных назад в базу. Опять таки мы прибегаем к помощи кнопки Submit, которой присваиваем имя, чтобы при втором проходе страницы проверить, какую часть кода нам надо выполнять.

Также мы здесь используем слегка измененное SQL-выражение.

```

<html>
<body>
    <?php
        $db = mysql_connect("localhost", "root");
        mysql_select_db("mydb",$db);

        if ($id) {
            if ($submit) {
                $sql = "UPDATE employees SET first='$first',last='$last',
                address='$address',position='$position' WHERE id=$id";

                $result = mysql_query($sql);
                echo "Thank you! Information updated.\n";
            } else {

```

```

        // query the DB
        $sql = "SELECT * FROM employees WHERE id=$id";
        $result = mysql_query($sql);
        $myrow = mysql_fetch_array($result);
    ?>

    <form method="post" action="<?php echo $PHP_SELF?>">
        <input type="hidden" name="id" value="<?php echo
        $myrow["id"] ?>">
        First name:<input type="Text" name="first" value="<?php
        echo $myrow["first"] ?>"><br>
        Last name:<input type="Text" name="last" value="<?php
        echo $myrow["last"] ?>"><br>
        Address:<input type="Text" name="address" value="<?php
        echo $myrow["address"] ?>"><br>
        Position:<input type="Text" name="position" value="<?php
        echo $myrow["position"] ?>"><br>
        <input type="Submit" name="submit" value="Enter
        information">
    </form>

<?php
}
} else {
    // display list of employees
    $result = mysql_query("SELECT * FROM employees",$db);
    while ($myrow = mysql_fetch_array($result)) {
        printf("<a href=\"%s?id=%s\">%s %s</a><br>\n",
        $PHP_SELF, $myrow["id"], $myrow["first"],
        $myrow["last"]);
    }
}
?>
</body>
</html>

```

Вот так. Нам удалось вставить все, что мы знаем и умеем в один код. Здесь вы можете увидеть, как мы используем выражение `if()` внутри другого выражения `if()` для последовательно проверки нескольких условий.

Теперь пришло время свести все вместе.

```

<html>
<body>
<?php
    $db = mysql_connect("localhost", "root");
    mysql_select_db("mydb",$db);

    if ($submit) {
        // here if no ID then adding else we're editing
        if ($id) {
            $sql = "UPDATE employees SET first='$first',last='$last',
            address='$address',position='$position'
            WHERE id=$id";
        } else {
            $sql = "INSERT INTO employees (first,last,address,position)
            VALUES
            ('$first','$last','$address','$position')";

```

```

    }
    // run SQL against the DB
    $result = mysql_query($sql);
    echo "Record updated/edited!<p>";
} else if ($delete) {
    // delete a record
    $sql = "DELETE FROM employees WHERE id=$id";
    $result = mysql_query($sql);
    echo "$sql Record deleted!<p>";
} else {
    // this part happens if we don't press submit
    if (!$id) {
        // print the list if there is not editing
        $result = mysql_query("SELECT * FROM employees",$db);
        while ($myrow = mysql_fetch_array($result)) {
            printf("<a href=\"%s?id=%s\">%s %s</a> \n", $PHP_SELF,
                $myrow["id"], $myrow["first"],
                $myrow["last"]);
            printf("<a href=\"%s?id=%s&delete=yes\">(DELETE)</a><br>",
                $PHP_SELF, $myrow["id"]);
        }
    }
}
?>
<P>
<a href="<?php echo $PHP_SELF?>">ADD A RECORD</a>
<P>
<form method="post" action="<?php echo $PHP_SELF?>">
    <?php
    if ($id)
    {
        // editing so select a record
        $sql = "SELECT * FROM employees WHERE id=$id";
        $result = mysql_query($sql);
        $myrow = mysql_fetch_array($result);
        $id = $myrow["id"];
        $first = $myrow["first"];
        $last = $myrow["last"];
        $address = $myrow["address"];
        $position = $myrow["position"];
        // print the id for editing
    }
    ?>
    <input type="hidden" name="id" value="<?php echo $id ?>">
    <?php
    }
    ?>
    First name:<input type="Text" name="first" value="<?php echo
    $first ?>"><br>
    Last name:<input type="Text" name="last" value="<?php echo
    $last ?>"><br>
    Address:<input type="Text" name="address" value="<?php echo
    $address ?>"><br>
    Position:<input type="Text" name="position" value="<?php echo
    $position ?>"><br>
    <input type="Submit" name="submit" value="Enter information">

```

```
</form>
<?php
}
?>
</body>
</html>
```

На первый взгляд код выглядит сложным, однако это не так. Программа делится на три части. Первое `if()` выражение проверяет, была ли нажата кнопка Submit, и если была, проводится проверка, есть ли в поданных данных переменная `$id`. Если ее нет, значит происходит добавление новой записи. В противном случае мы редактируем уже существующую запись.

Далее мы проверяем, определена ли переменная `$delete`. Если да, мы удаляем запись. Обратите внимание, что в первом выражении `if()` мы проверяем переменную, которая была подана с помощью метода POST, а в данном `if()` выражении мы проверяем переменную, которая является частью данных отправленных с помощью метода GET.

Наконец, мы переходим к действию, которое будет выполняться по умолчанию: то есть выводим просто список служащих и форму. Здесь мы опять проверяем существование переменной `$id`. Если она существует, мы просим базу данных выдать сведения о выбранном служащем. В противном случае выводим пустую форму.

Все, что мы рассмотрели, мы поместили в один большой код. Мы использовали циклы `while()` и выражения `if()`, а также целую гамму основных команд языка SQL — `SELECT`, `INSERT`, `UPDATE`, и `DELETE`.

Наконец, мы рассмотрели, как можно передавать информацию от одной страницы к другой через URL с помощью ссылок и через формы.